MODULE 03 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Cluster architecture and the life of a kubectl apply

Every component, what it does, and how they cooperate without talking to each other.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 3 of 35

admin@sthithapragna.com

Kubernetes is **a database of desired state**, and **a crowd of controllers** working to make reality match it.

admin@sthithapragna.com

The thermostat: a control loop

01

Desired

You set the thermostat to 21 degrees.

02

Observe

The sensor reads 18 degrees.

03

Compare

Three degrees short.

04

Act

Turn on the heating. Then observe again.

ON THE EXAM

A controller never finishes. It loops forever: observe, compare, act. Kubernetes calls this reconciliation.

The shape of a cluster

kubectl and clients
Users, CI, tools
Outside the cluster

etcd
All cluster state
Only the API server uses it

cloud-controller-mgr
Talks to your cloud
Optional

kube-scheduler
Places new Pods
Control plane

kube-apiserver
REST API over HTTPS
Authenticates and authorises
Scales horizontally
The only door to cluster state.

controller-manager
Built-in controllers
Control plane

kube-proxy
Service network rules
Every node, optional

kubelet
Runs Pods on its node
Every node

THE TAKEAWAY

Hub and spoke: components watch the API server for changes. They do not call each other.

Control plane components

COMPONENT	ITS ONE JOB
kube-apiserver	Exposes the Kubernetes API. Front end of the control plane.
etcd	Consistent, highly available key-value store for all cluster data.
kube-scheduler	Watches for new Pods with no node, and picks a node for each.
kube-controller-manager	Runs the built-in controllers in one process.
cloud-controller-manager	Links the cluster to a cloud provider's API.

REMEMBER

Scheduler places Pods. Controllers create them. The API server stores everything via etcd. Keep the verbs straight.

Controllers inside the controller manager

CONTROLLER	WATCHES FOR...	THEN...
`Deployment`	Deployments	Creates and scales ReplicaSets
`ReplicaSet`	ReplicaSets	Creates or deletes Pods to match replicas
`Node`	Node heartbeats	Marks unreachable nodes and evicts their Pods
`Job`	Jobs	Creates Pods and tracks completions
`EndpointSlice`	Services and ready Pods	Lists which Pod IPs back each Service
`ServiceAccount`	New namespaces	Creates the default ServiceAccount

ON THE EXAM

When a question asks what updates a node's Ready condition, or replaces a deleted Pod, the answer is a controller.

Node components

COMPONENT	ITS JOB
kubelet	Agent on every node. Makes sure the containers in its Pods' specs are running and healthy.
Container runtime	containerd or CRI-O. Actually runs the containers, via the CRI.
kube-proxy	Programs network rules so Service traffic reaches the right Pods. Optional with some CNI plugins.
Add-ons	Not core, but near universal: CNI plugin, CoreDNS, metrics-server.

ON THE EXAM

The kubelet only manages containers that Kubernetes created. It ignores anything started by hand on the node.

Control plane node versus worker node

CONTROL PLANE NODE

Decides

- Runs API server, etcd, scheduler, controllers
- Holds and changes cluster state
- Usually tainted so apps do not land there
- Several of them for high availability

WORKER NODE

Runs the apps

- Runs kubelet, kube-proxy, a runtime
- Hosts your Pods
- Reports status to the API server
- Every cluster needs at least one

THE TIEBREAKER

Control plane nodes manage state. Worker nodes run workloads. Managed services hide the control plane from you entirely.

Quayside deploys its booking API

```
$ kubectl apply -f booking-api.yaml

# booking-api.yaml
apiVersion: apps/v1
kind: Deployment
metadata: {name: booking-api}
spec:
  replicas: 3
  selector: {matchLabels: {app: booking-api}}
  template:
    metadata: {labels: {app: booking-api}}
    spec:
      containers:
      - {name: api, image: quayside/booking-api:2.4.1}
```

1 One command. We will follow what happens next, step by step.

8 The desired state: three copies. Nothing here says how or where.

The journey, part 1: from kubectl to Pods that exist

01
kubectl
Sends the object to the API server over HTTPS.

›

02
API server
Authenticates, authorises, runs admission, validates.

›

03
etcd
Stores the Deployment. kubectl reports created.

›

04
Deployment controller
Sees it and creates a ReplicaSet.

›

05
ReplicaSet controller
Creates three Pods, with no node yet.

ON THE EXAM

kubectl returns as soon as the object is stored. Nothing is running yet.

The journey, part 2: from Pods to traffic

01

Scheduler

Sees unassigned Pods. Filters, scores, binds each to a node.

›

02

kubelet

Sees a Pod bound to its node.

›

03

Runtime

Pulls the image, starts the containers via the CRI.

›

04

kubelet reports

Pod status goes back to the API server.

›

05

EndpointSlice

Ready Pods are added; kube-proxy updates rules.

ON THE EXAM

Pending means not yet scheduled or still pulling. Running means the kubelet has started the containers.

No component told another what to do. Each one **watched the API server** and reacted to what it saw.

When a component fails, what do you see?

IF THIS IS DOWN...	YOU SEE...
`kube-apiserver`	Running Pods keep running. No kubectl, no changes, no rescheduling.
`etcd`	The API server cannot read or store state. Lose it without a backup and the cluster is gone.
`kube-scheduler`	New Pods stay Pending with no node. Existing Pods are fine.
`controller-manager`	Nothing is replaced or scaled. Deleted Pods stay deleted.
`kubelet on one node`	That node goes NotReady; its Pods are later evicted and replaced elsewhere.

ON THE EXAM

Match the symptom to the one job. Pending with no node assigned points at the scheduler.

The cloud-controller-manager

- **Node controller.** Asks the cloud whether a silent node's VM still exists, and removes it if not.
- **Route controller.** Sets up routes in the cloud network between nodes.
- **Service controller.** Creates a cloud load balancer for each Service of type LoadBalancer.
- **No cloud integration?** A LoadBalancer Service's external IP stays pending forever.

> ON THE EXAM
>
> On-premises clusters have no cloud-controller-manager. They need something like MetalLB for LoadBalancer Services.

Quayside's system: the running example

PART	KUBERNETES OBJECT	WHY THAT OBJECT
Web front end	Deployment + Service	Stateless, many identical copies
Booking API	Deployment + autoscaler	Stateless, load varies by hour
Payment worker	Deployment	Long-running queue consumer
Ticket validator	Deployment	Go binary, distroless image
Bookings database	StatefulSet + volumes	Needs stable identity and storage
Log agent	DaemonSet	One copy on every node
Nightly manifest export	CronJob	Runs on a schedule, then finishes
Partner fare plugins	Pods + RuntimeClass	Untrusted code, sandboxed

ON THE EXAM

Every module from here uses this system. When a scenario mentions a part, you already know its shape.

> Quayside builds a test cluster with kubeadm on virtual machines in a public cloud. Nobody configures any cloud provider integration.
>
> They create a Service of type LoadBalancer for the web front end. Its external IP shows pending, and stays pending.

Which component's absence explains this?

A kube-proxy, which programs the Service rules on each node

B CoreDNS, which gives the Service its DNS name in the cluster

C kube-scheduler, which places the Service's Pods on nodes

D cloud-controller-manager, which talks to the cloud's API

◆ PAUSE AND DECIDE

Which component's absence explains this?

A kube-proxy, which programs the Service rules on each node

Handles traffic inside the cluster. It never creates cloud load balancers.

B CoreDNS, which gives the Service its DNS name in the cluster

DNS names are internal. The external IP comes from the cloud.

C kube-scheduler, which places the Service's Pods on nodes

Services are not scheduled, and Pod placement is unrelated.

D cloud-controller-manager, which talks to the cloud's API

Its service controller creates the cloud load balancer and fills in the IP.

THE SIGNAL

'No cloud integration' plus 'external IP pending' is the cloud-controller-manager, every time.

> After a control plane restart, Quayside scales the booking API from 3 to 6 replicas. kubectl shows the ReplicaSet now wants 6, and 6 Pods exist.
>
> The 3 new Pods stay Pending. They have no node assigned and no scheduling events. The 3 original Pods serve traffic normally.

Which component is most likely not working?

A kube-controller-manager

B etcd

C kube-scheduler

D The kubelet on every node

◆ PAUSE AND DECIDE

Which component is most likely not working?

A kube-controller-manager

It is working: the ReplicaSet controller created the three new Pods.

B etcd

State is being stored and read, so etcd is fine.

C kube-scheduler

Pods exist but no node was ever chosen: the scheduler's one job.

D The kubelet on every node

Kubelets act only after a Pod is bound to their node. None was.

THE SIGNAL

Pods exist but have no node: scheduler. Pods do not exist at all: controller manager.

> Quayside's security review wants a firewall rule so that only one Kubernetes component can connect to the etcd cluster.
>
> The cluster must keep working normally afterwards.

Which component should be allowed to reach etcd?

A kube-apiserver

B kube-controller-manager

C The kubelet on each node

D kube-scheduler

◆ PAUSE AND DECIDE

Which component should be allowed to reach etcd?

A kube-apiserver

The only component that reads and writes etcd. Everything else goes through it.

B kube-controller-manager

Controllers read and write state through the API server, not etcd.

C The kubelet on each node

Kubelets talk to the API server. Opening etcd to every node would be a big risk.

D kube-scheduler

The scheduler watches and binds through the API server too.

THE SIGNAL

Everything goes through the API server. etcd has exactly one client.

> During a control plane upgrade, Quayside's API server is unreachable for ten minutes. All worker nodes stay healthy.
>
> The on-call engineer asks what is happening to the booking API Pods already running on the workers.

What best describes those Pods during the outage?

A They stop, because the kubelet cannot confirm their spec

B They keep running, but nothing can be changed or replaced

C The scheduler moves them to nodes with a working API

D The kubelet deletes them after missing status updates

◆ PAUSE AND DECIDE

What best describes those Pods during the outage?

A They stop, because the kubelet cannot confirm their spec

The kubelet keeps running what it already has. It does not need permission.

B They keep running, but nothing can be changed or replaced

Workloads carry on; changes, scaling and healing wait for the API.

C The scheduler moves them to nodes with a working API

The scheduler needs the API server too, and never moves running Pods.

D The kubelet deletes them after missing status updates

Nothing deletes healthy Pods because reporting failed.

THE SIGNAL

The control plane decides; nodes run. A control plane outage freezes change, not workloads.

Module 3 in five lines

- Kubernetes is **desired state plus controllers** that reconcile, like a thermostat.
- **Only the API server talks to etcd.** Everything else watches the API server.
- **Scheduler places, controllers create, kubelet runs.** Keep the verbs straight.
- **Pending with no node** means scheduler. No LoadBalancer IP means cloud-controller-manager.
- A control plane outage **freezes change, not running workloads**.

Every object you apply has the same skeleton. Next: **the API and the object model**.